RELEASE / 19 OCTOBER 2026

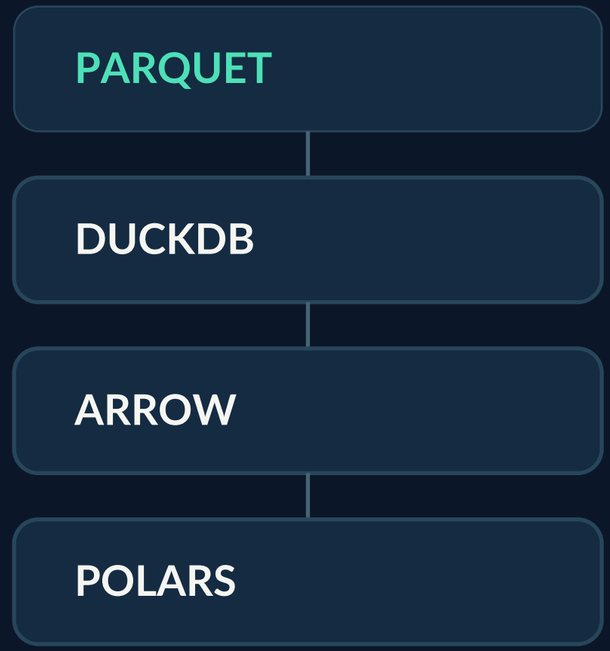
Familiar calls.

Modern analytics.

intermine314

0.2.0

Legacy InterMine workflows, shared execution and a resource-aware columnar pipeline.



One shared runtime, end to end.

Keep the query workflow. Move analytics into native columnar engines.

InterMine
Query / templates

›

Typed batches
Validate / convert

›

Parquet
Persist / compact

›

DuckDB
Filter / aggregate

›

Arrow + Polars
Read / analyze

Python coordinates transport, compatibility and ownership. Polars, DuckDB and Arrow do the native analytical work.

CSV INGESTION AND SPAWNED PARQUET TRANSFORMS ALSO SUPPORTED

Legacy workflows, canonical execution.

Change the package import and retain the historical service entry point.

```
from intermine314.webservice import Service

query = service.select("Gene.symbol")

query.export("genes.parquet")
```

Queries and expressions · saved XML · templates · lists and summaries · identifier jobs · registry helpers

460

historical symbols inventoried

331 passing scopes; 129 tested departures. Coverage is scoped, not universal equivalence.

Native Service keeps dictionaries; the historical facade keeps legacy row/object defaults.

A CPU budget with stage-specific workers.

Default native CPU budget: 16. Smaller profiles: 8, 6, 4 and 2.

4

Public-mine HTTP

Per-host permits

4 / 2

Arrow CPU / I/O

Separate native pools

16

DuckDB SQL

One main native query

12

Polars

One streaming plan

4

Coarse Python tasks

Spawned path-based work

1

Mutable resource

One owner per target

Allocations are stage choices, not additive pools. Actual CPU visibility, engine initialization and server limits still matter.

Keep work charged until it is consumed.

Pending, completed and emitting pages all belong to the outstanding-page envelope.

FETCH

Pending requests

WAIT

Completed pages

EMIT

Ordered consumption

Reservation remains active across the whole page lifecycle

Shared immutable row layouts reduce repeated metadata. Sized decompressed XML reads and bounded upload/transform queues reduce avoidable buffering.

Byte caps are estimates, not RSS guarantees; one oversized page may proceed to avoid deadlock.

Three concrete failure modes addressed.

Accept validated data; preserve previous output when work fails.

01

Server status

A page ending in wasSuccessful=false is rejected before its rows are accepted.

02

Compressed XML

Sized reads decompress response bytes before metadata reaches the XML parser.

03

Failed exports

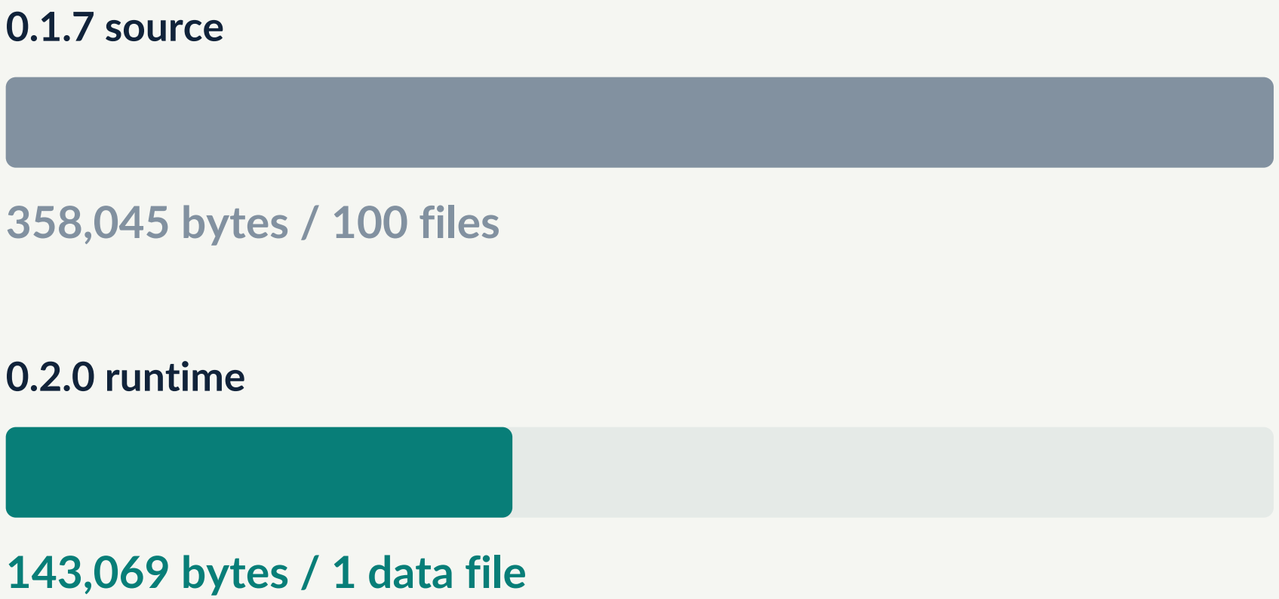
Staging and validation preserve the existing Parquet output after producer failure.

Publication is guarded per target; portable directory fallback is not a crash/cross-process transaction.

[A1] Fresh deterministic probes against published 0.1.7 source

Smaller Parquet. Far fewer small files.

100,000-row integer/string fixture; total bytes include the candidate manifest.



60%

less total storage

100 → 1

data files

Compaction trades write time for layout: this fixture took 75% more time. Savings depend on the dataset.

[A1] Matched modern dependencies / 21 paired local trials

One materialization. One response.

10,000 dictionary rows × 20 fields, on identical modern dependencies.

31%

less elapsed time

18.96 ms → 13.02 ms

2 → 1

requests

Unsized streams remove the extra count request during list(results).

Network latency is excluded. Result iterators no longer support len(); use explicit query counts when needed.

[A1] Median of 21 paired processes / local in-memory response

Focused improvements across the pipeline.

Development uplift: timing baseline 8e584c5; retained-allocation comparison uses the phase 1 baseline.

3.19×

Keyed-row access

10k rows × 20 fields

3.36×

Eligible numeric export

Single staged numeric part

9.11×

Repeated small SQL

Opt-in borrowed connection

84.7%

Less retained Python allocation

Shared layout; phase 1 → 2

Python allocation is not RSS. SQL setup reuse is opt-in. Whole-job gains depend on stage costs and input data.

[A2] Timing: 8e584c5; allocation: phase 1 → 2 / neither is PyPI 0.1.7

Fewer requests. Less repeated setup.

Opt-in workflow demonstrations with simulated 5 ms request latency; equal results checked.

32 → 4

Object reads

Batch prefetch; 180.2 → 22.9 ms

40 → 2

Service reuse

Reuse metadata; 222.1 → 11.1 ms

4 mines

Concurrent discovery

Same 12 requests; 67.4 → 19.1 ms

Prefetch is explicit. Cache initialization and resource lifetime remain guarded; mutations retain one owner.

Match the path to the workload.

Published 0.1.7 source comparison; identical modern dependencies, local fixtures.

Workload	0.1.7	0.2.0	Time change
Dictionary materialization	18.96 ms	13.02 ms	**-31%**
Dictionary streaming	8.65 ms	11.69 ms	**+35%**
Custom numeric single-file export	9.49 ms	21.69 ms	**+129%**
Custom directory export	59.56 ms	104.40 ms	**+75%**

These measurements do not establish a universal speedup or memory reduction. Directory compaction improves layout while adding write work.

Validated from source to installed wheel.

The matching tag triggers exact-commit checks before verified artifacts reach PyPI.

2,813

Source tests

Full local suite + API trace

904

Installed regressions

Source-package imports excluded

6

Installation modes

Wheel/sdist, extras and docs

Version consistency → master ancestry → exact artifact ID → SHA-256 verification → trusted-publisher upload

Validated platform: Linux CPython 3.14.5+. Other OSes and free-threaded builds remain outside certification.

A clear upgrade contract.

Use the migration guide alongside the familiar API surface.

01 Choose the service profile

Historical webservice import: legacy defaults. Native service import: dictionaries.

02 Move pandas workflows to Polars

dataframe() returns Polars; update package imports and dependency declarations.

03 Review stream and helper changes

Unsized streams; immutable streamed metadata; removed old policy/Tor helper modules.

04 Upgrade the Python environment

Python 3.14.5+; modern engine minima; PyArrow is a core dependency.

AVAILABLE ON PYPI

Start with your existing query.

Adopt the modern pipeline one workflow at a time.

```
python -m pip install intermine314==0.2.0
```

1. Retain familiar service and query calls.
2. Export validated data to Parquet.
3. Use DuckDB and Polars for repeated analysis.

pypi.org/project/intermine314/0.2.0

karikris/intermine314 · Python 3.14.5+ · MIT AND BSD-2-Clause

Every headline has a named baseline.

Measurements are local observations, not public-server capacity or whole-job guarantees.

[A1] Published 0.1.7 comparison + migration

Matched modern dependencies; 21 pairs; fixtures, memory limits and raw Parquet.

[A2] Development uplift + phase 2 memory comparison

Separate 21-pair timings; shared-layout retained allocation; opt-in SQL reuse.

[A3] Opt-in workflow demonstrations

Simulated 5 ms request latency; prefetch, discovery and Service reuse.

[A4] Exact release workflow + publication evidence

Tagged commit, six installation modes, installed regressions and artifact hashes.

Benchmark host: WSL2, 18 visible CPUs; CPython 3.14.5; JIT off. No attribution to Python 3.14 itself: both sides used the same interpreter.